🐼 pandas

pandas is a powerful, open-source library built on top of Python, specifically designed for **data manipulation and analysis**. The name "pandas" is derived from "panel data," an econometrics term for datasets that include observations over multiple time periods for the same individuals.

It provides high-performance, easy-to-use data structures and data analysis tools. The two primary data structures in pandas are:

1. `Series`: A one-dimensional labeled array capable of holding any data type (integers, strings, floating-point numbers, Python objects, etc.). It's like a single column in a spreadsheet.
2. `DataFrame`: A two-dimensional labeled data structure with columns of potentially different types. It's the most commonly used pandas object and represents a rectangular table, like a spreadsheet or a SQL table.

Why Use pandas?

pandas is the go-to tool for data wrangling and analysis in Python for several key reasons:

- **Simple Data Handling**: It makes it incredibly easy to load, manipulate, and save data from various formats.
- **Powerful Data Structures**: The DataFrame object is intuitive and flexible, allowing you to work with tabular data seamlessly.
- **Rich Functionality**: It provides a huge set of tools for:
  - **Reading/Writing Data**: Effortlessly read from and write to formats like CSV, Excel, SQL databases, JSON, and more.
  - **Data Cleaning**: Handle missing data (`NaN`), drop or fill values, and remove duplicates.
  - **Data Transformation**: Reshape, pivot, merge, and concatenate datasets.
  - **Selection & Slicing**: Easily select subsets of your data using labels (`.loc`) or integer positions (`.iloc`).
  - **Analysis & Aggregation**: Perform powerful "group by" operations, calculate descriptive statistics, and run aggregations.
  - **Time Series**: Excellent, integrated support for time-series data.
- **Performance**: It is highly optimized for performance, with many of its core operations written in C or Cython.
- **Integration**: It integrates perfectly with other libraries in the data science ecosystem, such as **NumPy**, **Matplotlib** (for plotting), and **Scikit-learn** (for machine learning).

Essential Methods & Attributes

Here are some of the most fundamental attributes and methods you will use constantly. (Assume `df` is a pandas DataFrame and `pd` is the imported alias `import pandas as pd`).

Talha Shahid

1. Creating & Loading Data

- `pd.DataFrame(data_dict)`: Create a DataFrame from a Python dictionary.
- `pd.read_csv('file.csv')`: Load data from a CSV file.
- `pd.read_excel('file.xlsx')`: Load data from an Excel file.
- `df.to_csv('output.csv')`: Save a DataFrame to a CSV file.

2. Inspecting Data

- `df.head(n)`: View the first `n` rows (default is 5).
- `df.tail(n)`: View the last `n` rows (default is 5).
- `df.info()`: Get a concise summary of the DataFrame, including data types and non-null counts.
- `df.describe()`: Get descriptive statistics (count, mean, std, min, max, etc.) for numerical columns.
- `df.shape`: Get the dimensions of the DataFrame (rows, columns) as a tuple.
- `df.columns`: View the column labels (names).
- `df.values`: Get the data as a NumPy array.

3. Selecting Data (Indexing & Slicing)

- `df['column_name']`: Select a single column as a `Series`.
- `df[['col1', 'col2']]`: Select multiple columns as a new `DataFrame`.
- `df.loc[row_label, col_label]`: Select data by **labels** (names of rows/columns).
  - `df.loc[0]`: Selects row with label 0.
  - `df.loc[0:5, ['col1', 'col2']]`: Selects rows 0 through 5, for columns 'col1' and 'col2'.
- `df.iloc[row_index, col_index]`: Select data by **integer position** (index).
  - `df.iloc[0]`: Selects the first row (at index 0).
  - `df.iloc[0:5, [0, 1]]`: Selects the first 5 rows and the first 2 columns.

4. Data Cleaning

- `df.dropna()`: Remove rows with any missing values (`NaN`).
- `df.fillna(value)`: Fill all missing values with a specific `value`.
- `df.rename(columns={'old_name': 'new_name'})`: Rename columns.
- `df.drop_duplicates()`: Remove duplicate rows.

- `df.drop(columns=['col_to_drop'])` : Drop a specific column.

5. Analysis & Operations

- `df['col'].value_counts()` : Count the occurrences of unique values in a column.
- `df.groupby('column_name')` : Group the DataFrame by one or more columns. This is often followed by an aggregation.
  - `df.groupby('Category')['Price'].mean()` : Calculates the average 'Price' for each 'Category'.
- `df.sort_values(by='column_name')` : Sort the DataFrame by values in a column.
- `pd.merge(df1, df2, on='key')` : Combine two DataFrames based on a common key (like a SQL JOIN).
- `pd.concat([df1, df2])` : Stack two DataFrames on top of each other (axis=0) or side-by-side (axis=1).
- `df.apply(function)` : Apply a function along an axis (rows or columns).

Creating a DataFrame from a Dictionary (Columns)

When you pass a Python dictionary to `pd.DataFrame()`, pandas uses a specific and very common structure:

- **Dictionary Keys**: Each `key` in the dictionary becomes a **column name** in the DataFrame.
- **Dictionary Values**: Each `value` associated with a key must be a list (or another array-like structure, like a pandas Series or NumPy array). The elements in this list become the **data for that column**.

Crucial Rule: All the lists provided as values *must have the same length*. This length determines the number of rows in the new DataFrame.

Here you have 4 keys ("`Book Title`", "`Author`", "`Pice (PKR)`", "`Stock`"), so you will get 4 columns. Each key has a list of 4 elements, so you will get 4 rows.

Example Code

```
import pandas as pd

data = {"Book Title": ["The Alchemeist", "Forty Rules of Love", "Sapiens", "Atomic Habits"],
        "Author": ["Paulo Coelho", "Elif Shafak", "Yuval Noah Harari", 'James Clear'],
        "Pice (PKR)": [1200, 1500, 2200, 1800],
        "Stock": [15, 25, 10, 30]}

# Create the DataFrame
df = pd.DataFrame(data)

# Display the DataFrame
df
```

Talha Shahid

In [72]:
```
import pandas as pd

data = {"Book Title": ["The Alchemeist", "Forty Rules of Love", "Sapiens", "Atomic Habits"],
        "Author": ["Paulo Coelho", "Elif Shafak", "Yuval Noah Harari", 'James Clear'],
        "Pice (PKR)": [1200, 1500, 2200, 1800],
        "Stock": [15, 25, 10, 30]}

df = pd.DataFrame(data)
```

In [73]:
```
df
```

Out[73]:

	Book Title	Author	Pice (PKR)	Stock
0	The Alchemeist	Paulo Coelho	1200	15
1	Forty Rules of Love	Elif Shafak	1500	25
2	Sapiens	Yuval Noah Harari	2200	10
3	Atomic Habits	James Clear	1800	30

Adding a **Series**

A `pd.Series` is a one-dimensional, labeled array. It's the fundamental data structure for a single column (or row) in pandas. Think of it as a Python list but with an associated **index**, which acts as a set of labels for each data point.

General Syntax

The basic syntax for creating a Series is:

```
pd.Series(data=None, index=None, name=None, dtype=None)
```

In this case:

- **`data`** : You provided the list `["Fiction", "Romance", "History", "Self-help"]` .
- **`index`** : `index` **`is not`** provided. Therefore, pandas assigned the default 0-based `RangeIndex` .
- **`name`** : `name` is also not provided, so the Series is currently unnamed.

In [74]:
```
genre = pd.Series(["Fiction", "Romance", "History", "Self-help"])
```

In [75]:
```
genre
```

Out[75]:
```
0      Fiction
1      Romance
2      History
3    Self-help
dtype: object
```

In [76]:
```
df['Genre'] = genre
```

In [77]:
```
df
```

Out[77]:

	Book Title	Author	Pice (PKR)	Stock	Genre
0	The Alchemeist	Paulo Coelho	1200	15	Fiction
1	Forty Rules of Love	Elif Shafak	1500	25	Romance
2	Sapiens	Yuval Noah Harari	2200	10	History
3	Atomic Habits	James Clear	1800	30	Self-help

Dropping a Column

To remove a column from a DataFrame, you use the `.drop()` method. The most straightforward way to drop a column is to use the `columns` parameter.

Syntax

The general syntax is:

```
df.drop(columns=['column_name1', 'column_name2'])
```

In [78]:
```
df.drop(columns='Genre')
```

Out[78]:

	Book Title	Author	Pice (PKR)	Stock
0	The Alchemeist	Paulo Coelho	1200	15
1	Forty Rules of Love	Elif Shafak	1500	25
2	Sapiens	Yuval Noah Harari	2200	10
3	Atomic Habits	James Clear	1800	30

Talha Shahid

Important: `inplace` vs. Returning a Copy

By default, `.drop()` does not change your **original** DataFrame. It returns a **new DataFrame** with the column removed.

If you want to modify your original df directly, you have two options:

1. **Reassign the variable (Recommended):** This is the preferred and clearer method.

In [79]:
```
# The original 'df' is replaced by the new one
df = df.drop(columns=['Genre'])
```

In [80]:
```
df
```

Out[80]:

	Book Title	Author	Pice (PKR)	Stock
0	The Alchemeist	Paulo Coelho	1200	15
1	Forty Rules of Love	Elif Shafak	1500	25
2	Sapiens	Yuval Noah Harari	2200	10
3	Atomic Habits	James Clear	1800	30

2. **Use `inplace=True`:** This modifies the original df in place and returns `None`.

In [81]:
```
df['Genre'] = pd.Series(["Fiction", "Romance", "History", "Self-help"])

df.drop(columns=['Genre'], inplace=True)
```

In [82]:
```
df
```

Out[82]:

	Book Title	Author	Pice (PKR)	Stock
0	The Alchemeist	Paulo Coelho	1200	15
1	Forty Rules of Love	Elif Shafak	1500	25
2	Sapiens	Yuval Noah Harari	2200	10
3	Atomic Habits	James Clear	1800	30

Adding a Column with .insert()

While the most common way to add a column is by simple assignment (e.g., `df['new_col'] = myList`), this method always adds the new column to the **end** of the DataFrame.

The `.insert()` method gives you precise control over **where** the new column is placed.

Syntax

The general syntax is:

```
df.insert(loc, column, value)
```

1. **`loc`:** The integer position (index) where you want to insert the new column. `loc=0` would make it the first column, `loc=1` the second, and so on.
2. **`column`:** The string name you want to give the new column.
3. **`value`:** The data for the column. This can be a `pd.Series`, a list, a NumPy array, or a single value (which will be repeated for all rows).

Important: Modifies in Place

Unlike `.drop()` (which returns a new **copy** by default), `.insert()` **modifies** your original DataFrame in place and returns None.

This means you do not reassign the variable.

Correct:

```
df.insert(2, 'Genre', genre)
```

Incorrect:

```
df = df.insert(2, 'Genre', genre) # (This will make df equal to None)
```

In [83]:
```
df.insert(loc=2, column="Genre", value=genre)
```

In [84]:
```
df
```

Out[84]:

	Book Title	Author	Genre	Pice (PKR)	Stock
0	The Alchemeist	Paulo Coelho	Fiction	1200	15
1	Forty Rules of Love	Elif Shafak	Romance	1500	25
2	Sapiens	Yuval Noah Harari	History	2200	10
3	Atomic Habits	James Clear	Self-help	1800	30

Filtering the DataFrame

1. **Check Stock**

In [85]:
```
bookData = df[df['Book Title'] == 'Sapiens']
```

`df['Book Title'] == 'Sapiens'` is a boolean condition. This will result in a dataframe have boolean values. `True` assigns to rows where the title matches and `False` to the rows where there is no match.

In [86]:
```
df['Book Title'] == 'Sapiens'
```

Out[86]:
```
0    False
1    False
2     True
3    False
Name: Book Title, dtype: bool
```

`df[df['Book Title'] == 'Sapiens']` this will create a `df` of those rows only where the value is `True`

In [87]:
```
df[df['Book Title'] == 'Sapiens']
```

Out[87]:

	Book Title	Author	Genre	Pice (PKR)	Stock
2	Sapiens	Yuval Noah Harari	History	2200	10

In [88]:
```
if not bookData.empty:
    # Get the stock value from the first row of the filtered result
    stockCount = bookData['Stock'].iloc[0]
```

```
    print(f"Stock for 'Sapiens': {stockCount}")
else:
    print("Book not found")
```

```
Stock for 'Sapiens': 10
```

Full Code

In [89]:
```
def checkStock(bookTitle):
    # Create a new DataFrame by filtering for rows where 'Book Title' matches
    bookData = df[df['Book Title'] == bookTitle]

    # If the filtered DataFrame is not empty, a match was found
    if not bookData.empty:
        # Get the stock value from the first row of the filtered result
        stockCount = bookData['Stock'].iloc[0]
        print(f"Stock for '{bookTitle}': {stockCount}")
    else:
        # If the filtered DataFrame is empty, no match was found
        print(f"Book not found: '{bookTitle}'")

checkStock("Sapiens")
checkStock("The Hobbit")
```

```
Stock for 'Sapiens': 10
Book not found: 'The Hobbit'
```

2. **Calculate Total Inventory Value**

In [90]:
```
# Multiply the two columns and then sum the result
totalInventoryValue = (df['Pice (PKR)'] * df['Stock']).sum()

print(f"The total monetary value of the inventory is: {totalInventoryValue} PKR")
```

```
The total monetary value of the inventory is: 131500 PKR
```

3. **Generate a Reorder List**

In [91]:
```
# 1. Create a boolean mask for the condition
reOrderCondition = df['Stock'] <= 15

# 2. Apply the mask to filter the DataFrame and then select the 'Book Title' column
reorderList = df[reOrderCondition]['Book Title']

print("--- Reorder List (Stock <= 15) ---")
print(reorderList)
```

```
--- Reorder List (Stock <= 15) ---
0    The Alchemeist
2           Sapiens
Name: Book Title, dtype: object
```

In [92]:
```
df
```

Out[92]:

	Book Title	Author	Genre	Pice (PKR)	Stock
0	The Alchemeist	Paulo Coelho	Fiction	1200	15
1	Forty Rules of Love	Elif Shafak	Romance	1500	25
2	Sapiens	Yuval Noah Harari	History	2200	10
3	Atomic Habits	James Clear	Self-help	1800	30

Fetching Desired Rows & Columns

You can get particular rows and columns in pandas using two main methods: `.loc[]` and `.iloc[]`.

1. **`df.loc[]`** is for selecting by **label** (e.g., row index name, column name).
2. **`df.iloc[]`** is for selecting by **integer position** (e.g., 1st row, 3rd column).

1. Using `df.loc[]` (Label-Based)

This method uses the **index** labels (like `'a'`, `'b'`, `'c'`) and column names (like `'Author'`).

Syntax:

```
df.loc[row_labels, column_labels]
```

Get a specific cell Get the `'Author'` of the book at index label `'c'`.

In [93]:
```
author2 = df.loc[2, 'Author']
print("\nAuthor at index '2':")
print(author2)
```

```
Author at index '2':
Yuval Noah Harari
```

Get a specific row

In [94]:
```
row2 = df.loc[2]
print("\nRow 2:")
print(row2)
```

```
Row 2:
Book Title              Sapiens
Author        Yuval Noah Harari
Genre                   History
Pice (PKR)                 2200
Stock                        10
Name: 2, dtype: object
```

Get multiple rows and columns

Get the `'Book Title'` and `'Stock'` for rows `'0'` and `'3'`.

In [95]:
```
subset = df.loc[[0, 3], ['Book Title', 'Stock']]
print("\nSubset (rows '0', '3' and cols 'Book Title', 'Stock'):")
print(subset)
```

```
Subset (rows '0', '3' and cols 'Book Title', 'Stock'):
       Book Title  Stock
0  The Alchemeist     15
3   Atomic Habits     30
```

2. Using `df.iloc[]` (Integer-Position-Based)

This method uses the integer position `(0, 1, 2...)` for both rows and columns.

Syntax:

`df.iloc[row_positions, column_positions]`

Get a specific cell Get the data at the **2nd** row `(position 1)` and **5th** column `(position 4)`.

In [96]:
```
cell_1_2 = df.iloc[1, 4]
print("\nCell at row 1, col 4 (Stock for 40 Rules):")
print(cell_1_2)
```

```
Cell at row 1, col 4 (Stock for 40 Rules):
25
```

Get a specific row

Get the entire first row (position 0).

In [97]:
```
firstRow = df.iloc[0]
print("\nFirst row (position 0):")
print(firstRow)
```

```
First row (position 0):
Book Title    The Alchemeist
Author           Paulo Coelho
Genre                Fiction
Pice (PKR)              1200
Stock                     15
Name: 0, dtype: object
```

Get multiple rows and columns (using slices)

Get the **first two rows** `(positions 0 and 1)` and the **first two columns** `(positions 0 and 1)`.

In [98]:
```
# Slicing with iloc: 0:2 means "start at 0, up to but not including 2"
subsetIloc = df.iloc[0:2, 0:2]
print("\nFirst 2 rows and first 2 columns:")
print(subsetIloc)
```

```
First 2 rows and first 2 columns:
             Book Title        Author
0       The Alchemeist  Paulo Coelho
1  Forty Rules of Love   Elif Shafak
```